ANDROID

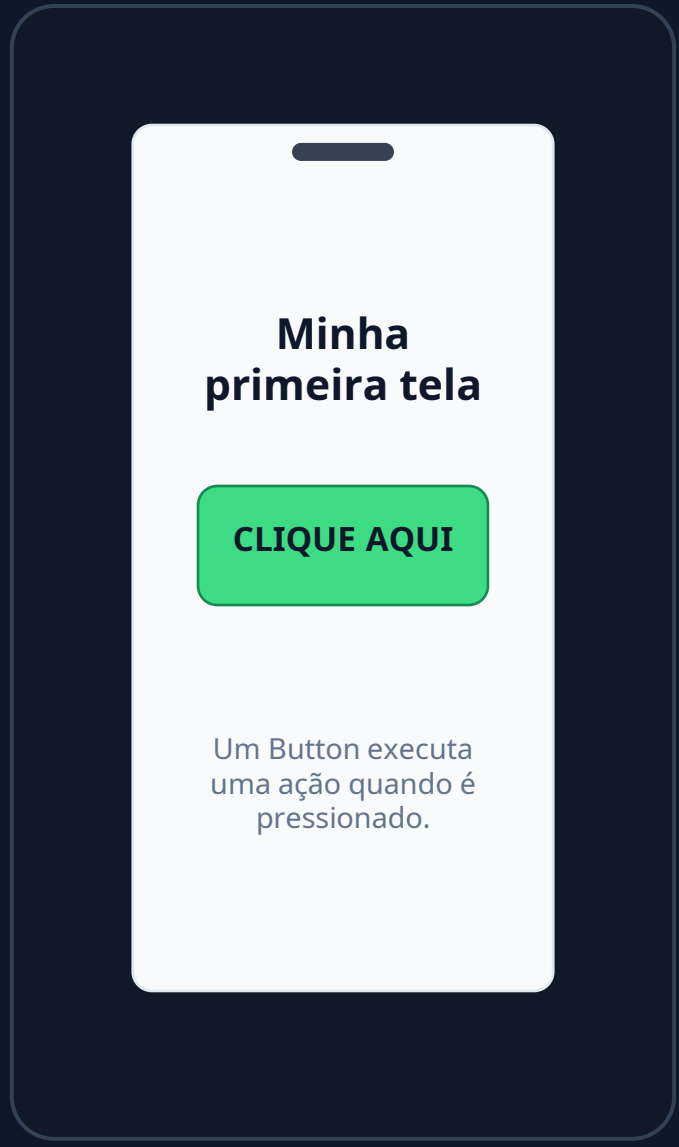
Componente Button

Criando ações a partir de cliques na interface

Aula introdutória

XML + Java

Programação para Dispositivos Móveis 1



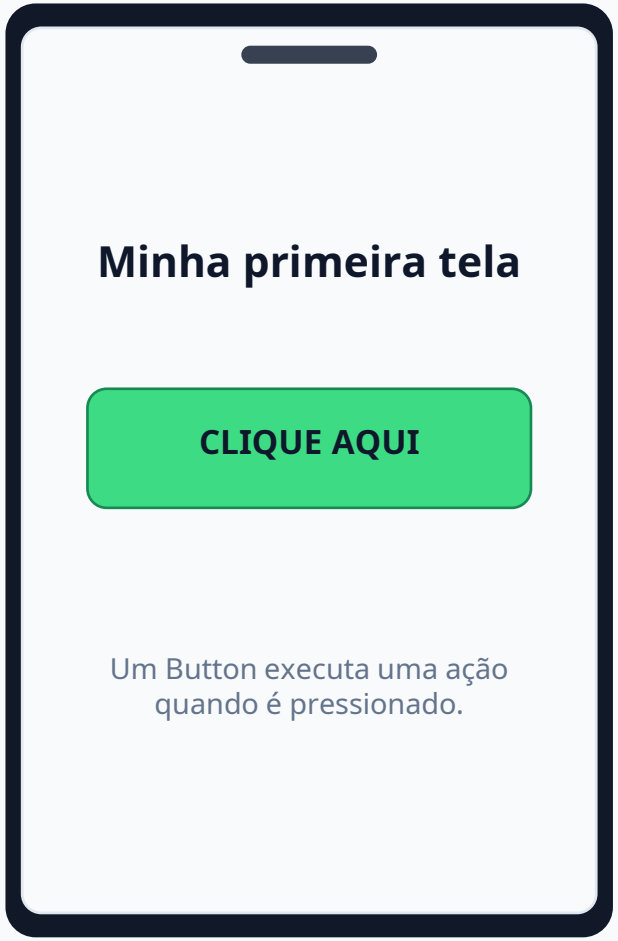
O que é o Button?

É o componente usado quando queremos que o usuário pressione algo para executar uma ação.

Button

- Exibe um texto que indica a ação, como “Salvar”, “Entrar” ou “Calcular”.
- Pode detectar o clique ou toque do usuário.
- Ao ser pressionado, normalmente executa um trecho de código Java.
- Assim como o TextView, possui propriedades de texto como text, textSize e textColor.

Ideia-chave: o Button normalmente representa uma ação que o usuário deseja realizar.



Adicionando um Button ao layout

Assim como os outros componentes, podemos usar o editor Design ou escrever o XML.

Pelo editor visual

1. Abra o arquivo activity_main.xml
2. Escolha a visualização Design
3. Localize Button na Palette
4. Arraste o componente para a tela

Diretamente no XML

```
<Button

android:layout_width="wrap_content"

android:layout_height="wrap_content"
    android:text="Clique aqui" />
```

O Android Studio permite alternar entre Design e Code para observar como o componente é representado no XML.

Primeiro Button no XML

Um exemplo simples com os atributos mais importantes.

```
<Button
    android:id="@+id/btnClique"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Clique aqui"
    android:textSize="18sp" />
```

Observe

- id identifica o Button no código Java.
- wrap_content ocupa apenas o espaço necessário.
- text define o texto mostrado no botão.
- textSize define o tamanho do texto.
- Depois precisamos definir o que ocorrerá quando ele for clicado.

Button básico

Largura e altura do Button

wrap_content e match_parent são valores fundamentais em layouts Android.

wrap_content

- O componente ocupa apenas o espaço necessário para mostrar seu conteúdo.
- Em um Button, normalmente significa espaço suficiente para o texto e o espaçamento interno.
- É muito usado para botões pequenos e simples.

```
android:layout_width="wrap_content"
android:layout_height="wrap_content"
```

match_parent

- O componente tenta ocupar todo o espaço disponível fornecido pelo elemento pai.
- É útil quando queremos um botão ocupando toda a largura disponível.
- Pode ser combinado com uma altura em wrap_content.

```
android:layout_width="match_parent"
android:layout_height="wrap_content"
```

Principais atributos do Button

Os atributos mais úteis para uma primeira aula.

Atributo	Para que serve
`android:id`	Identifica o botão para que possamos acessá-lo no Java.
`layout_width / height`	Define largura e altura: wrap_content, match_parent ou uma medida.
`android:text`	Define o texto exibido no botão.
`android:textSize`	Define o tamanho do texto, normalmente em sp.
`android:textColor`	Define a cor do texto.
`android:textStyle`	Pode aplicar normal, bold ou italic.
`android:enabled`	Define se o botão pode ou não ser pressionado.
`android:backgroundTint`	Permite alterar a cor de fundo do botão.

Texto direto ou strings.xml?

O texto do Button também pode ser armazenado como recurso.

Texto direto

```
android:text="Salvar"
```

- É simples para testes e exemplos rápidos.
- O Android Studio pode alertar que o texto está fixo no layout.
- Para projetos maiores, prefira armazenar o texto em strings.xml.

Usando strings.xml

```
<string name="salvar">Salvar</string>
```

```
android:text="@string/salvar"
```

A mesma regra usada no TextView vale para o texto exibido no Button.

Acessando o Button pelo Java

O id do XML permite localizar o componente dentro da Activity.

```
<Button
    android:id="@+id/btnClique"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Clique aqui" />
```



```
Button btnClique =

findViewById(R.id.btnClique);
```

O nome usado em R.id.btnClique deve corresponder ao android:id definido no XML.

Depois de localizar o Button, podemos configurar o evento de clique.

Detectando o clique do usuário

setOnClickListener() define o que será executado quando o Button for pressionado.

```
Button btnClique = findViewById(R.id.btnClique);

btnClique.setOnClickListener(v -> {
    // código executado quando o botão for clicado
});
```

`setOnClickListener()`

- “Escuta” o clique no botão.
- O código dentro de { } é executado quando o clique acontece.
- v representa a View que disparou o evento.

Para esta fase, a ideia principal é: Button → clique → execução de uma ação.

Exemplo: Button alterando um TextView

O Button dispara o evento; o TextView mostra o resultado.

```
TextView txtMensagem = findViewById(R.id.txtMensagem);
Button btnClique = findViewById(R.id.btnClique);

btnClique.setOnClickListener(v -> {
    txtMensagem.setText("Você clicou no botão!");
});
```

Fluxo

1. Usuário toca no Button
2. O listener percebe o clique
3. setText() altera o TextView

entrada de evento

saída em texto

Integrando Plain Text + Button + TextView

Um pequeno formulário reúne os três componentes vistos na aula.

```
EditText edtNome = findViewById(R.id.edtNome);
Button btnMostrar = findViewById(R.id.btnMostrar);
TextView txtResultado = findViewById(R.id.txtResultado);

btnMostrar.setOnClickListener(v -> {
    String nome = edtNome.getText().toString();
    txtResultado.setText("Olá, " + nome + "!");
});
```



Botão habilitado ou desabilitado

enabled controla se o usuário pode interagir com o Button.

Habilitado

```
android:enabled="true"
```

- Pode receber clique normalmente.
- É o estado padrão do Button.

```
btnSalvar.setEnabled(true);
```

Desabilitado

```
android:enabled="false"
```

- Não executa a ação ao ser pressionado.
- É útil enquanto algum dado obrigatório ainda não foi informado.

```
btnSalvar.setEnabled(false);
```

Personalizando a aparência

Alguns atributos do TextView também podem ser usados no Button.

```
<Button
    android:id="@+id/btnSalvar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Salvar"
    android:textSize="18sp"
    android:textStyle="bold"
    android:textColor="#000000"
    android:backgroundTint="#3DDC84" />
```

Atributos úteis

- textSize: tamanho do texto.
- textStyle: negrito ou itálico.
- textColor: cor do texto.
- backgroundTint: cor de fundo.
- layout_width: pode ocupar toda a largura.



Erros e dúvidas comuns

Detalhes simples que costumam aparecer nas primeiras atividades.

Esquecer o id	×	`sem android:id`	✓	`android:id="@+id/btnSalvar"`
id diferente no Java	×	`R.id.btnGravar`	✓	`usar exatamente o id do XML`
Esperar ação automática	×	`Button sem listener`	✓	`usar setOnClickListener()`
Confundir text e hint	×	`hint no Button`	✓	`Button usa normalmente text`
Usar dp no texto	×	`textSize="18dp"`	✓	`textSize="18sp"`

Exercício prático

Objetivo: criar uma pequena interface usando Plain Text, Button e TextView.

Faça no Android Studio

1. Adicione um Plain Text para o usuário digitar o nome.
2. Adicione um Button com o texto “Mostrar”.
3. Adicione um TextView vazio para apresentar o resultado.
4. Crie um id para cada componente.
5. No Java, localize os três componentes com findViewById().
6. No clique do Button, leia o nome digitado.
7. Mostre no TextView: “Olá, [nome]!”.

Desafio extra

Adicione um segundo Button chamado “Limpar”. Ao clicar nele:

- limpe o Plain Text;
- limpe o TextView;
- teste enabled=false em um dos botões.

Assim o aluno pratica entrada, ação e saída de dados na mesma tela.

Resumo da aula

O que o aluno deve saber ao final.

- ✓ Reconhecer o Button como componente que representa uma ação.
- ✓ Adicionar um Button pelo Design ou pelo XML.
- ✓ Configurar id, text, largura, altura e aparência básica.
- ✓ Entender claramente wrap_content e match_parent.
- ✓ Localizar o Button no Java com findViewById().
- ✓ Detectar cliques usando setOnClickListener().
- ✓ Combinar Plain Text, Button e TextView em uma pequena interface.

Conceito central

ENTRADA
Plain Text

AÇÃO
Button

SAÍDA
TextView

Programação para Dispositivos Móveis 1